Open Source trifft KI: PostgreSQL für moderne Vektor-Workloads

Daniel Seybold | benchANT

data2day

About me



benchANT

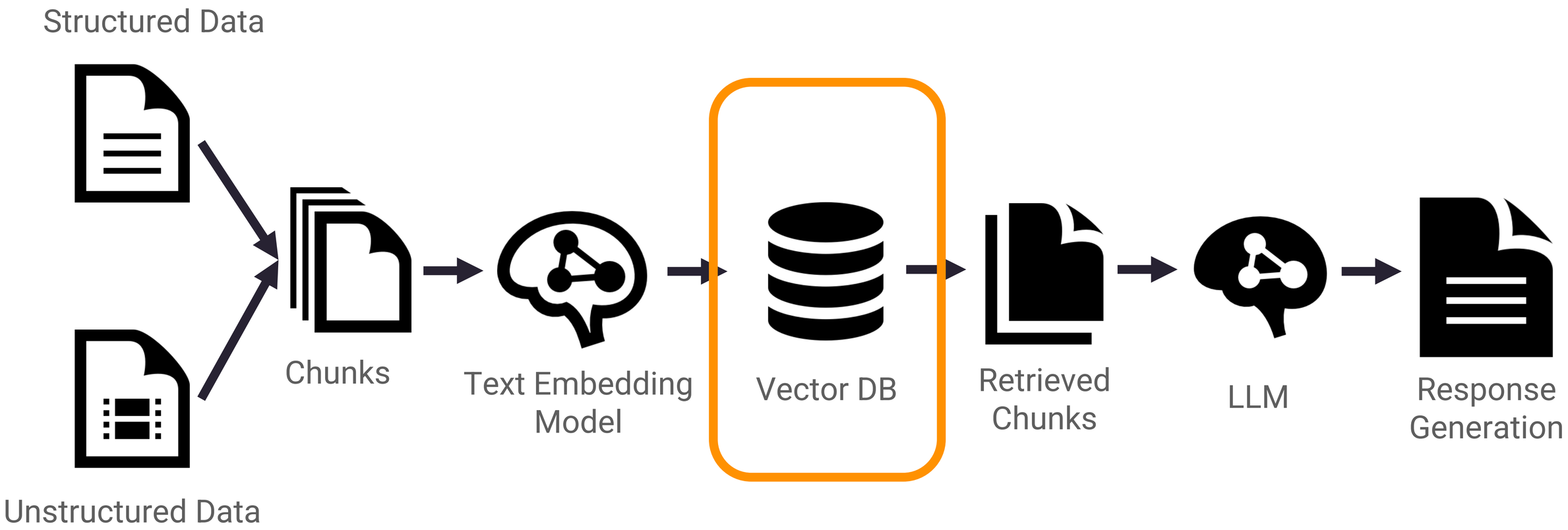
- Co-Founder of benchANT
- PhD about benchmarking cloud and database systems
- All about distributed systems, databases and cloud
- Enjoys to demystify the black art of database benchmarking
- Loves every kind of racket sports

benchANT

What is Driving the Database Landscape?

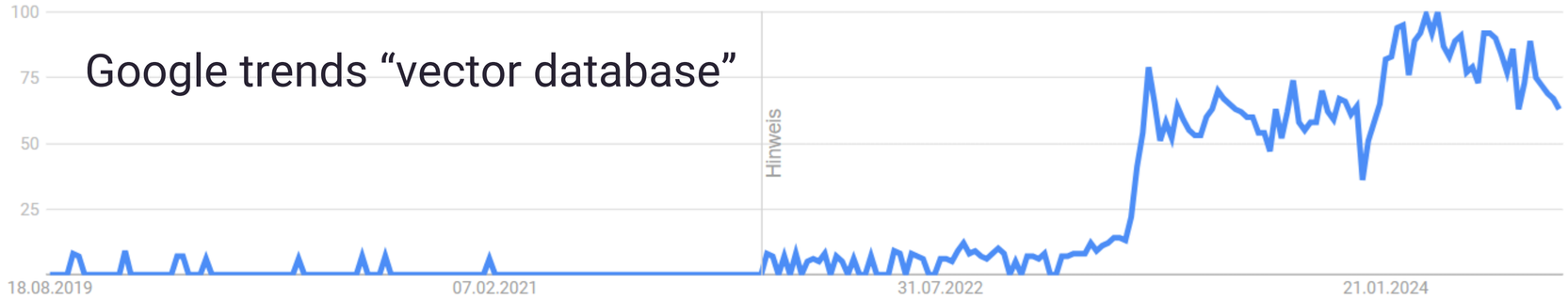
benchANT

A Basic RAG Architecture



benchANT

What is Driving the Database Landscape?





benchANT

The Vector Database Landscape

native vector databases

Vespa

Milvus

Weaviate

Pinecone

qdrant

General purpose databases with vector support

PostgreSQL

cassandra

MongoDB.

elastic

SingleStore

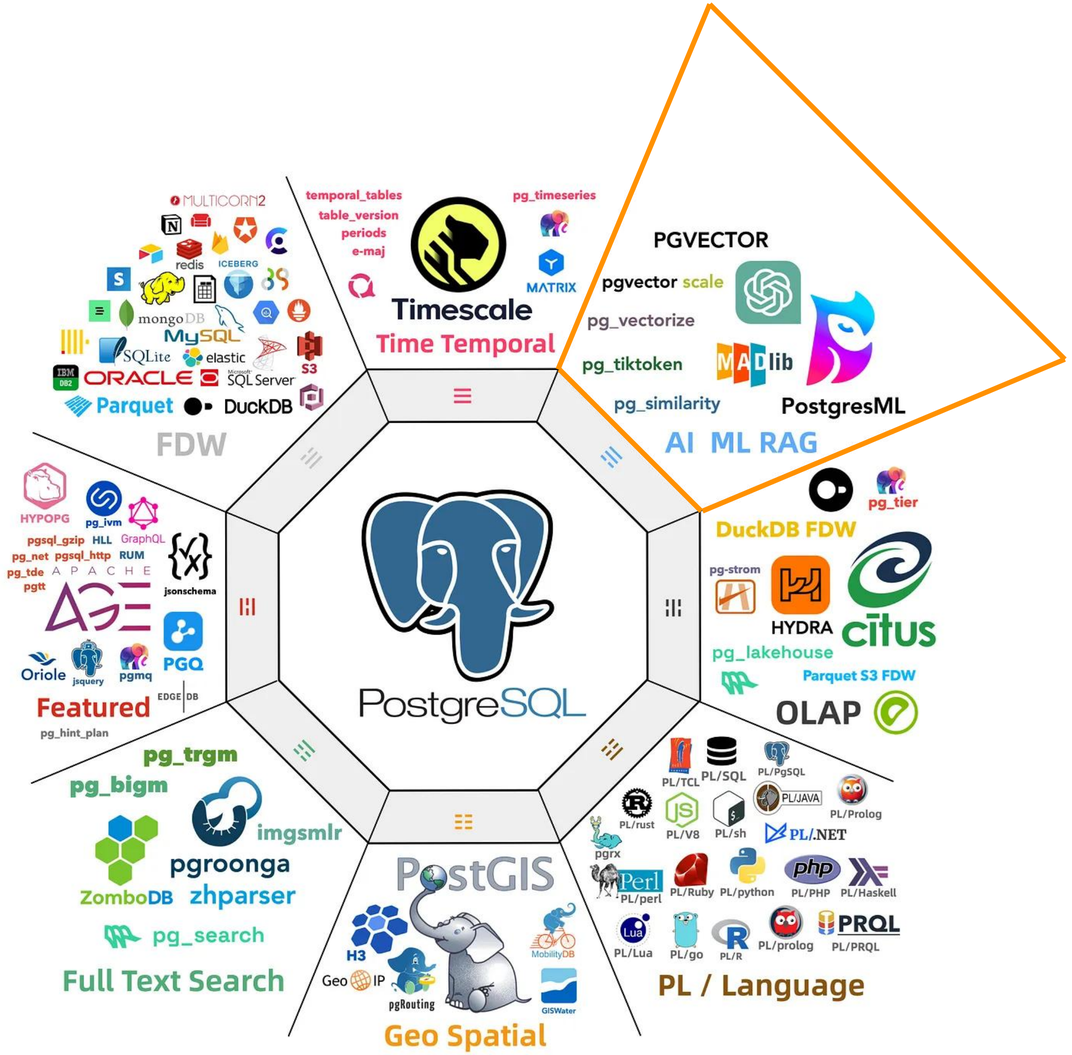
Couchbase

MySQL

ORACLE DATABASE

Microsoft SQL Server

The Elephant in the Room



https://www.timescale.com/blog/postgres-the-birdhorse-of-databases/

benchANT

PostgreSQL & Vector Search

benchANT

What is a Vector Database?

- support for storing embeddings, `m` vectors of `n` dimensions, representing a compressed, low-dimensional representation of data (text, video, audio)
- the higher the number of dimensions the better the quality
- embeddings are typically created outside of the database
- provides similarity search capabilities by algorithms of the Approximate Nearest Neighbor (ANN) class

benchANT

What is a Vector Database?

- vector data requires specialized index types to enable efficient vector search queries

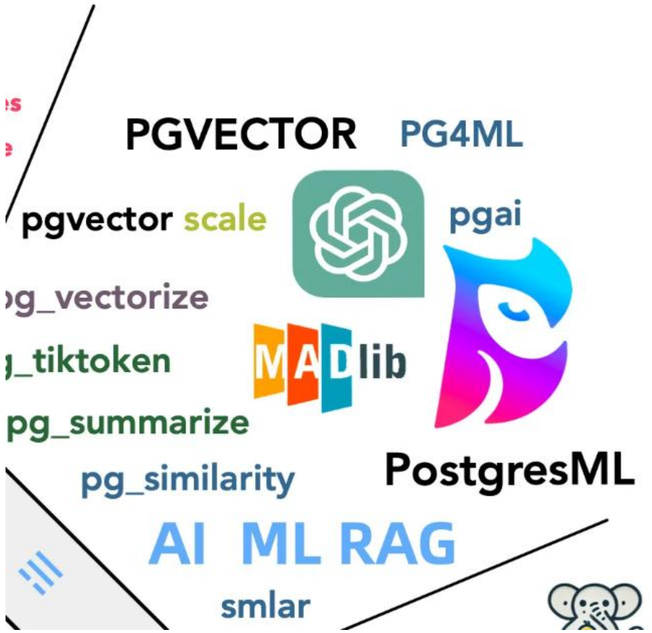
- various indexing algorithms are available
  - Inverted File Index (IVF)
  - Hierarchical Navigable Small World (HNSW) graphs
  - and many more

- for more details on vector databases see https://thedataquarry.com/posts/vector-db-1/

benchANT

PostgreSQL Vector Search Extensions

- pgvector (0.8.1)
- pgvecto.rs (0.4.0)
- VectorChord (0.5.3)
- pgvectorscale (0.8.0)
- lantern (0.5.0)
- and many more



benchANT

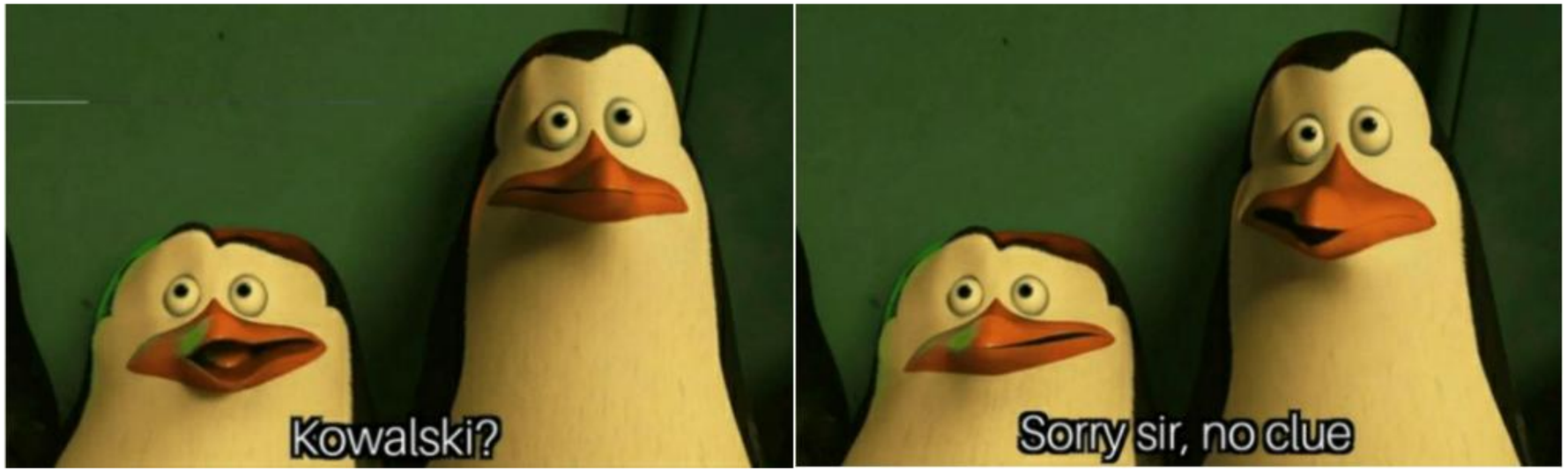
How do PostgreSQL Vector Search Extensions differentiate?

- index types (IVFFLAT, HNSW, DiskANN, …)
- index quantization options (binary, scalar, product, …)
- Additional index tuning options
- Supported dimensions, e.g. pgvecto.rs supports up to 65535 dimensions
- Unique features (e.g. available in cloud offers, dedicated index creation processes, filtering …)

benchANT

The Bigger the Choice…

- Which index type fits best for my target data set?
- How to tune my index and search queries for performance and precision?
- Which throughput and latency can be expected?
- Which PostgreSQL extension provides the best performance?
- Do I need a dedicated Vector Database?



benchANT

Blog Posts, Whitepapers, Rankings, Benchmarking, …

- Blog Posts
  - https://dmitry-kan.medium.com/the-rise-fall-and-future-of-vector-databases-how-to-pick-the-one-that-lasts-6b9fbb43bbbe
  - https://benchant.com/blog/single-store-vector-vs-pinecone-zilliz-2025
  - https://www.mongodb.com/docs/atlas/atlas-vector-search/tutorials/vector-search-quick-start
- Whitepapers: https://www.cockroachlabs.com/guides/vector-search-meets-distributed-sql
- Ranking: https://zilliz.com/vdbbench-leaderboard?dataset=vectorSearch
- Benchmarking: "One accurate measurement is worth a thousand expert opinions." - Grace Hopper

benchANT

Benchmarking Vector Databases

benchANT

Vector Benchmark Suites

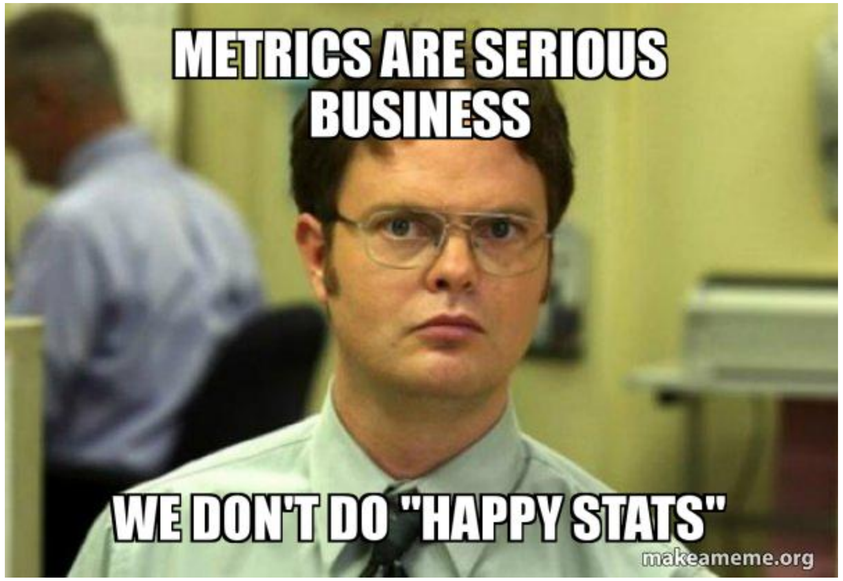
- ANN-Benchmark
- Big-ANN-Benchmark
- pgvectorbench
- Qdrant vector-db-benchmark
- **Zilliz VectorDBBench**

For a continuously updated list of database benchmarks: https://benchant.com/blog/benchmarking-suites

benchANT

Vector Database Benchmark Metrics

- throughput
  - ingestion
  - search
- latency
  - search
- performance-cost-ratio
- **recall — search quality**
- **index build time**



benchANT

PostgreSQL Vector Search Benchmarks

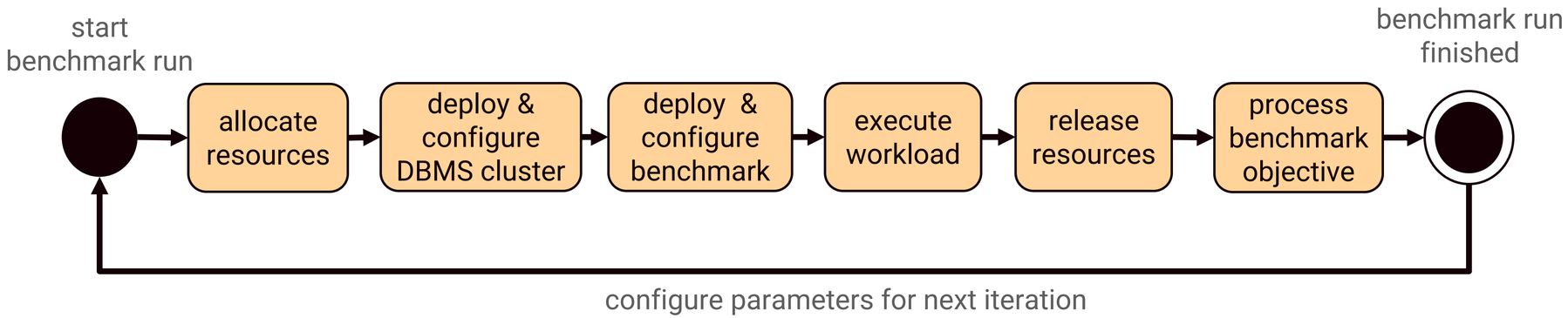
benchANT

Benchmark Objectives

- understanding the knobs and bolts to consider for vector search benchmarks
  - for PostgreSQL
  - in general, when running vector search benchmarks or reading reports
- performance impact of
  - Recall vs. throughput
  - data set characteristics
  - PostgreSQL vector search extensions
  - PostgreSQL vs. other general purpose and dedicated vector databases

Benchmark Methodology

- benchmarks are executed via the benchANT framework
  - automated database benchmarking in the cloud
  - https://oparu.uni-ulm.de/xmlui/handle/123456789/37430



benchANT

VectorDBBench Overview

- released and maintained by Zilliz (the company behind the open source vector database Milvus)
- active community and frequent releasees
- extensible architecture
- supports >15 vector databases
- comes with multiple data sets, from 500K to 100M vectors of different dimensions
- support for using custom data sets
- support for different uses cases: search, streaming, ingestion

benchANT

VectorDBBench Benchmark Workflow (Search)

```
def load():
# download vector data set
# create database
# ingest embeddings into database (single threaded)

def optimize():
# create index
# apply DB specific tuning (optional)

def query():
# execute single threaded search queries for recall
# execute concurrent search queries x threads over y minutes
```

benchANT

Benchmark Setup

- PostgreSQL Version 17 with a config by https://pgtune.leopard.in.ua/
  - pgvector 0.8.1
  - pgvecto.rs 0.4.0
  - pgvectorscale 0.8.0
- AWS EC2 `m7i.4xlarge` VM with 16 vCores/64GB RAM and 500GB GP3 storage
- VectorDBBench 1.0.5 forked by benchANT
  - minor CLI extension to integrate into benchANT framework
  - minor binding extensions to set all required parameters explicitly
- VectorDBBench is deployed on a `r7i.4xlarge` VM in the same region and AZ as the PostgreSQL instance

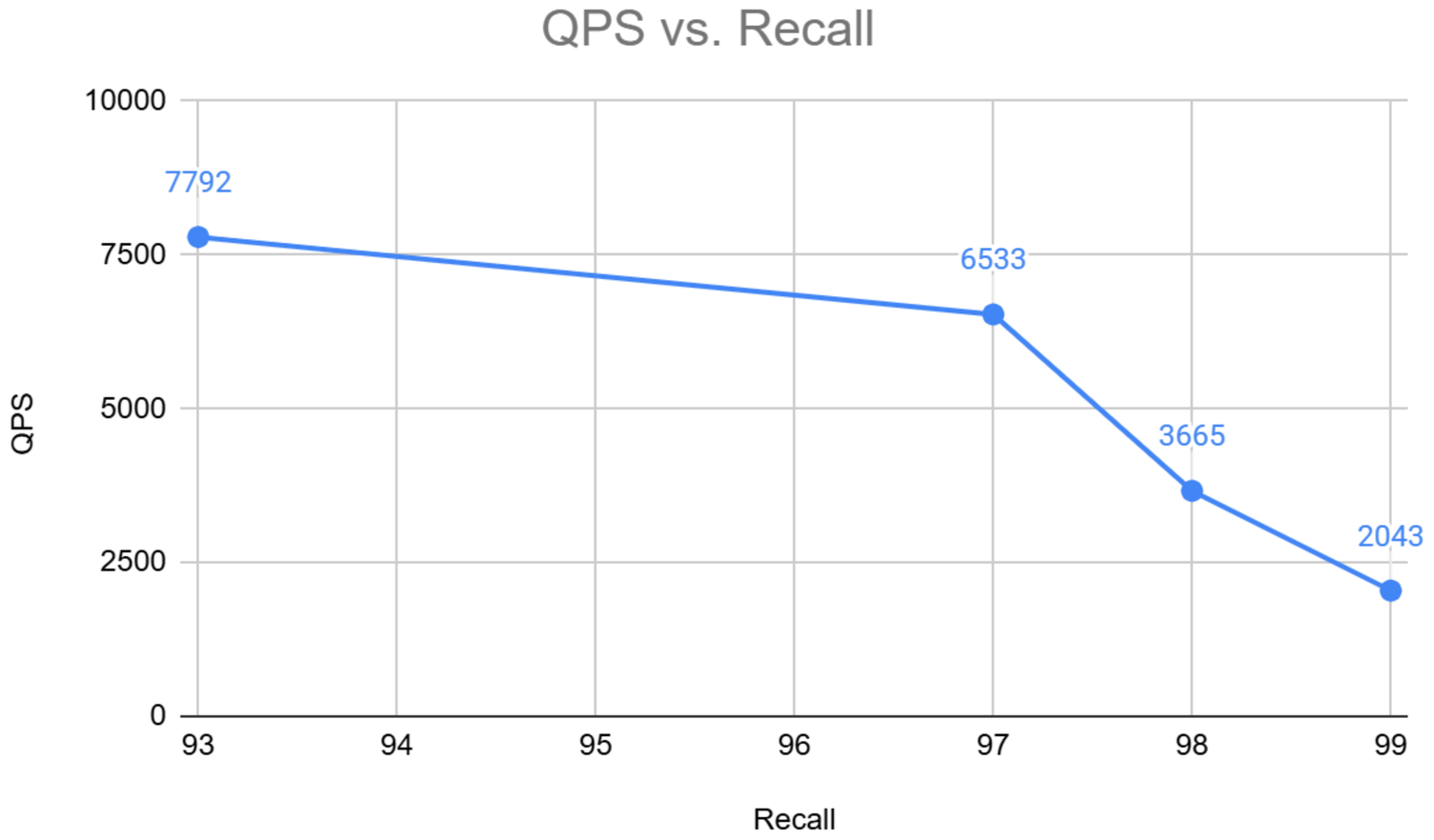
benchANT

Vector Index Types: Example HNSW

- Hierarchical Navigable Small World (HNSW) Graph, a graph-based index structure designed for Approximate Nearest Neighbor (ANN) search in high-dimensional vector spaces
- Builds a multi-layer graph:
  - Upper layers connect long-range neighbors for fast navigation
  - Lower layers capture dense, local neighborhoods for precision
  - Search starts from the top layer and progressively descends, refining candidates
- Key characteristics:
  - High recall at low latency
  - Memory-intensive
  - Widely adopted
- There are many more index types, e.g. https://milvus.io/docs/index-explained.md

benchANT

Cohere 1Mx768 — QPS vs. Recall pgvector (HNSW)

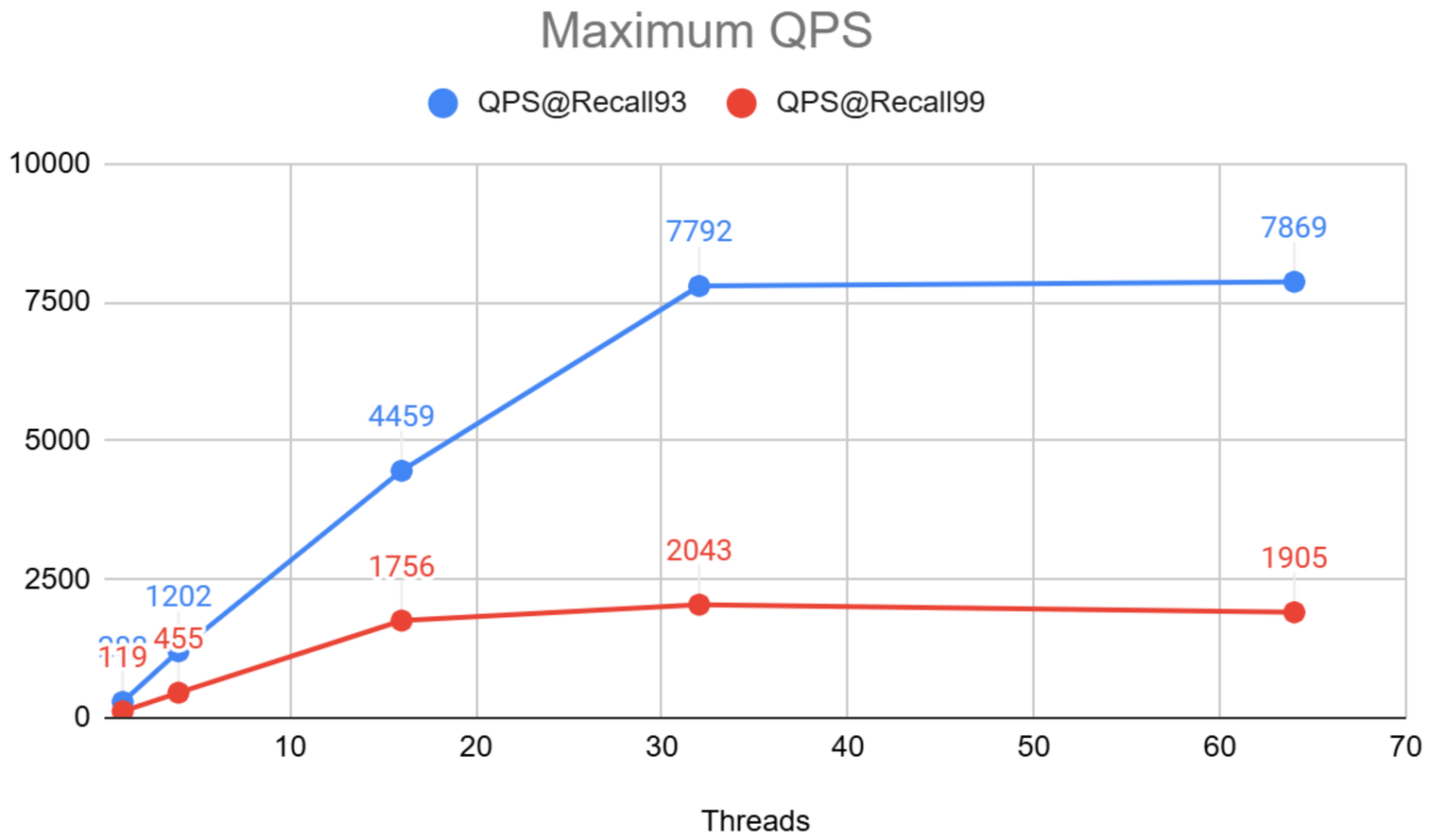


benchmark configuration

- PostgreSQL with pgvector
- index parameters
  - m=32
  - efConstrucion=128
  - quantization=none
- search parameters
  - efSearch={16,32,64,128}
  - threads=32

benchANT

Cohere 1Mx768 — Maximum QPS pgvector (HNSW)



benchmark configuration

- PostgreSQL with pgvector
- index parameters
  - m=32
  - efConstrucion=128
  - quantization=none
- search parameters
  - efSearch={16,128}
  - threads={1,4,16,32,64}

benchANT

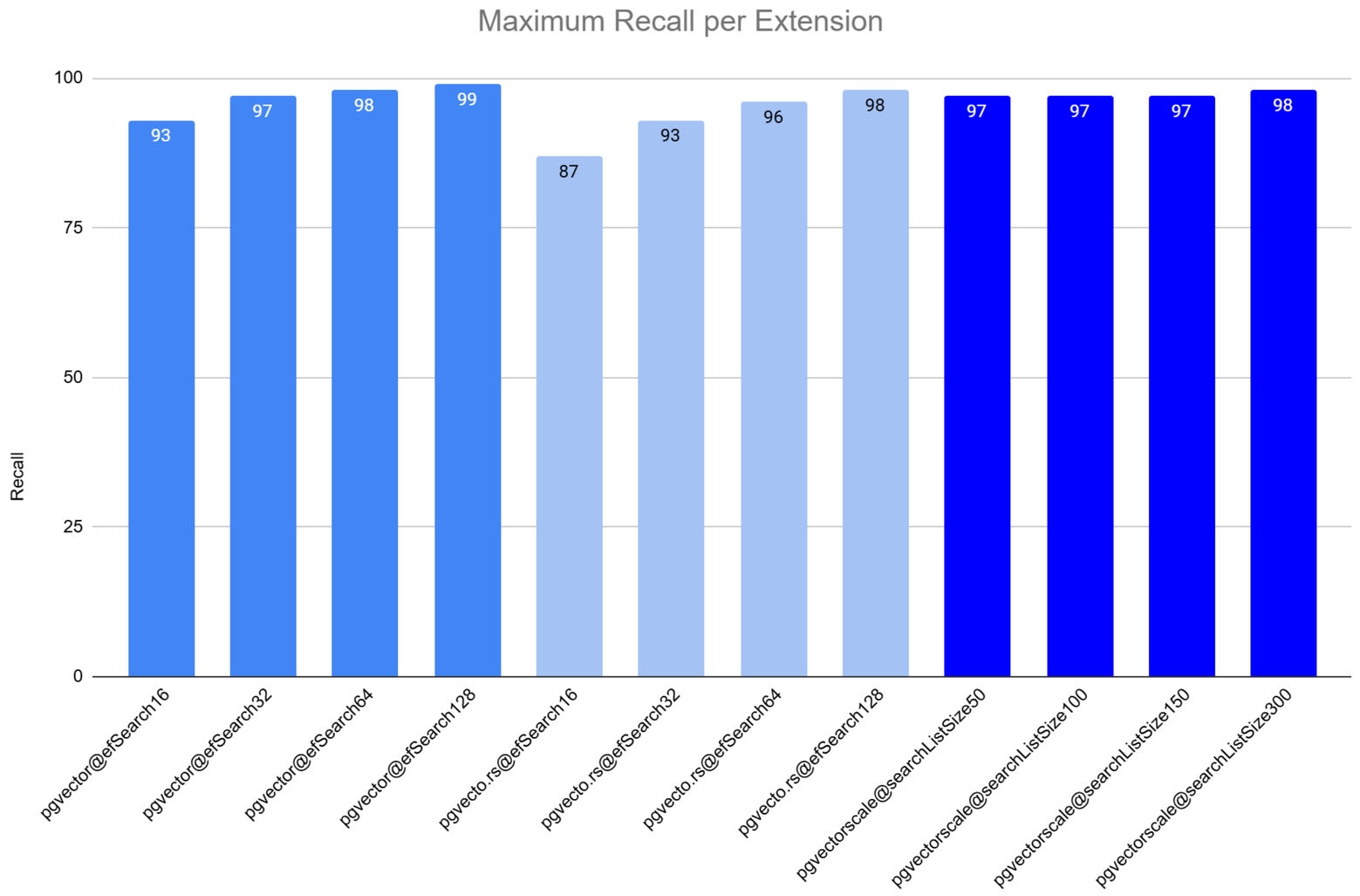
pgvector vs. pgvecto.rs vs. pgvectorscale (Cohere 1M)

Goal: Illustrate performance differences between PostgreSQL vector extensions without juding nor fully optimizing each implementation

		1M Cohre x 768				
PostgreSQL extension	**Index Type**	**Quantization**	**efConstruction**	**M**	**Index Size [GB]**	**Index Build Time [m]**
pgvector	HNSW	None	128	32	3.8	6
pgvecto.rs	HNSW	None	128	32	3.3	22
pgvectorscale	DiskANN	SBQ	128 (searchListSize)	64 (numNeighbors)	0.7	71

benchANT

Cohere 1Mx768 — Maximum Recall

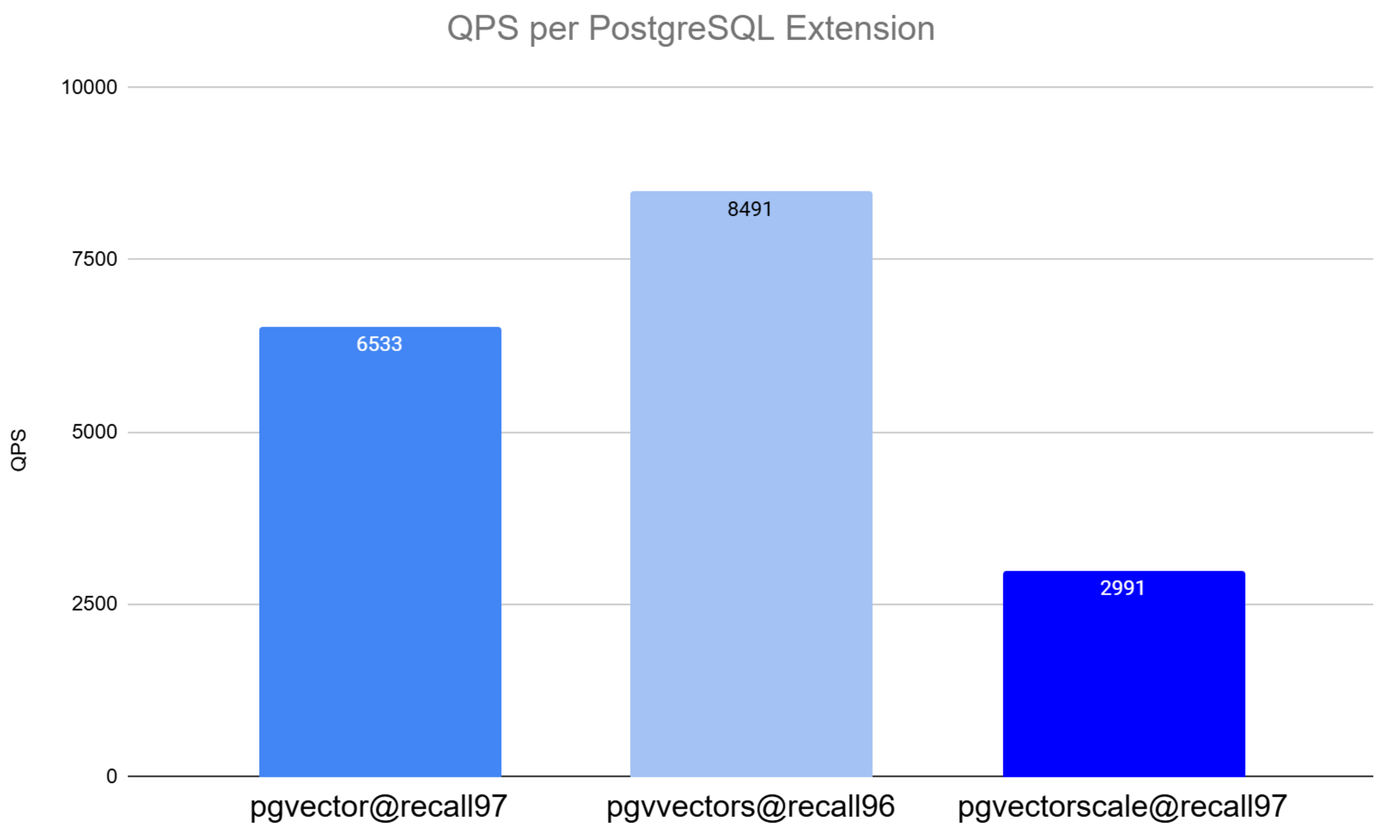


benchmark configuration

- PostgreSQL with `pgvector, pgvecto.rs, pgvectorscale`

benchANT

Cohere 1Mx768 — Maximum QPS



Might be worth to check the `pgvecto.rs` successor
https://github.com/tensorchord/VectorChord

benchmark configuration

- PostgreSQL with `pgvector`, `pgvecto.rs`, `pgvectorscale`
- comparable recall levels
- threads = 32

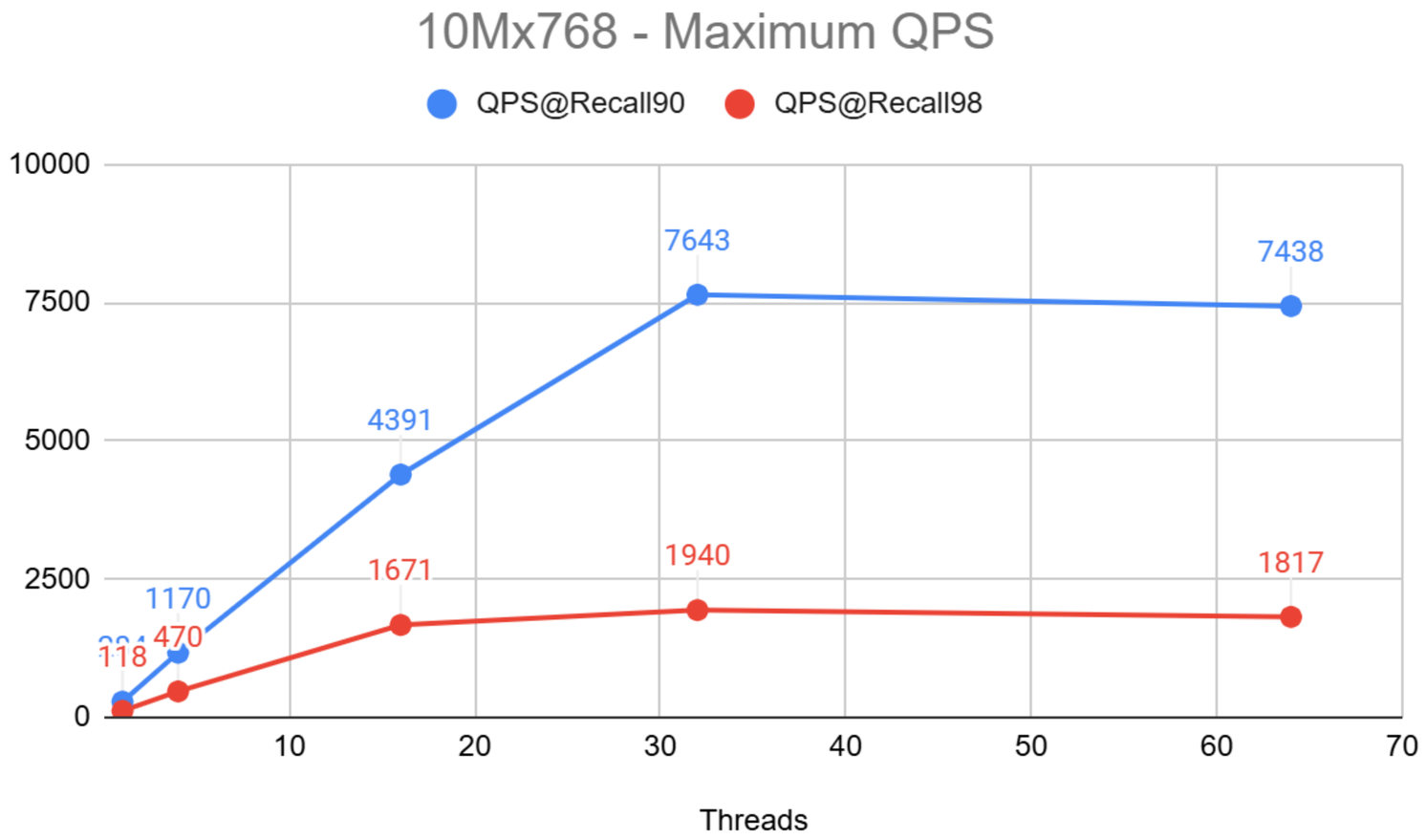
benchANT

From 1Mx768 to 10Mx768

- PostgreSQL configuration and instance size remain identical to the 1Mx768 data set
- HNSW index parameters (`pgvector`/`pgvecto.rs`) M and efConstruction remain identical to the 1Mx768 data set
- HNSW index is configured with quantization to reduce total index size
- DiskANN (`pgvectorscale`) index configurations remain identical to the 1Mx768 data set

benchANT

Cohere 10Mx768 — Maximum QPS pgvector (HNSW)

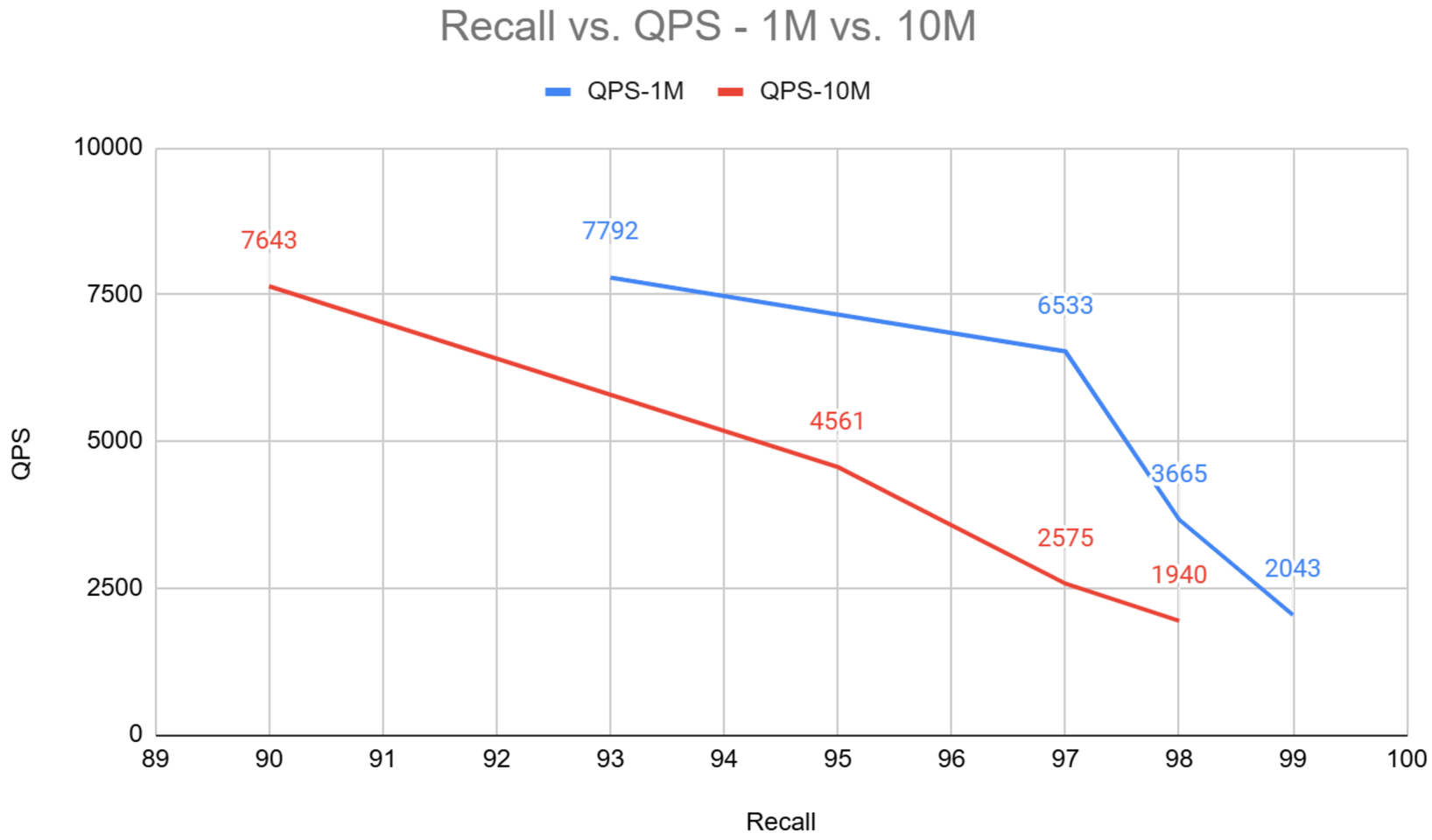


benchmark configuration

- PostgreSQL with pgvector
- index parameters
  - m=32
  - efConstrucion=128
  - quantization=halfvec
- search parameters
  - efSearch={16,128}
  - threads={1,4,16,32,64}

benchANT

Cohere 1Mx768 vs. 10Mx768 — QPS vs. Recall pvector (HNSW)



benchANT

pgvector vs. pgvecto.rs vs. pgvectorscale (Cohere 10M)

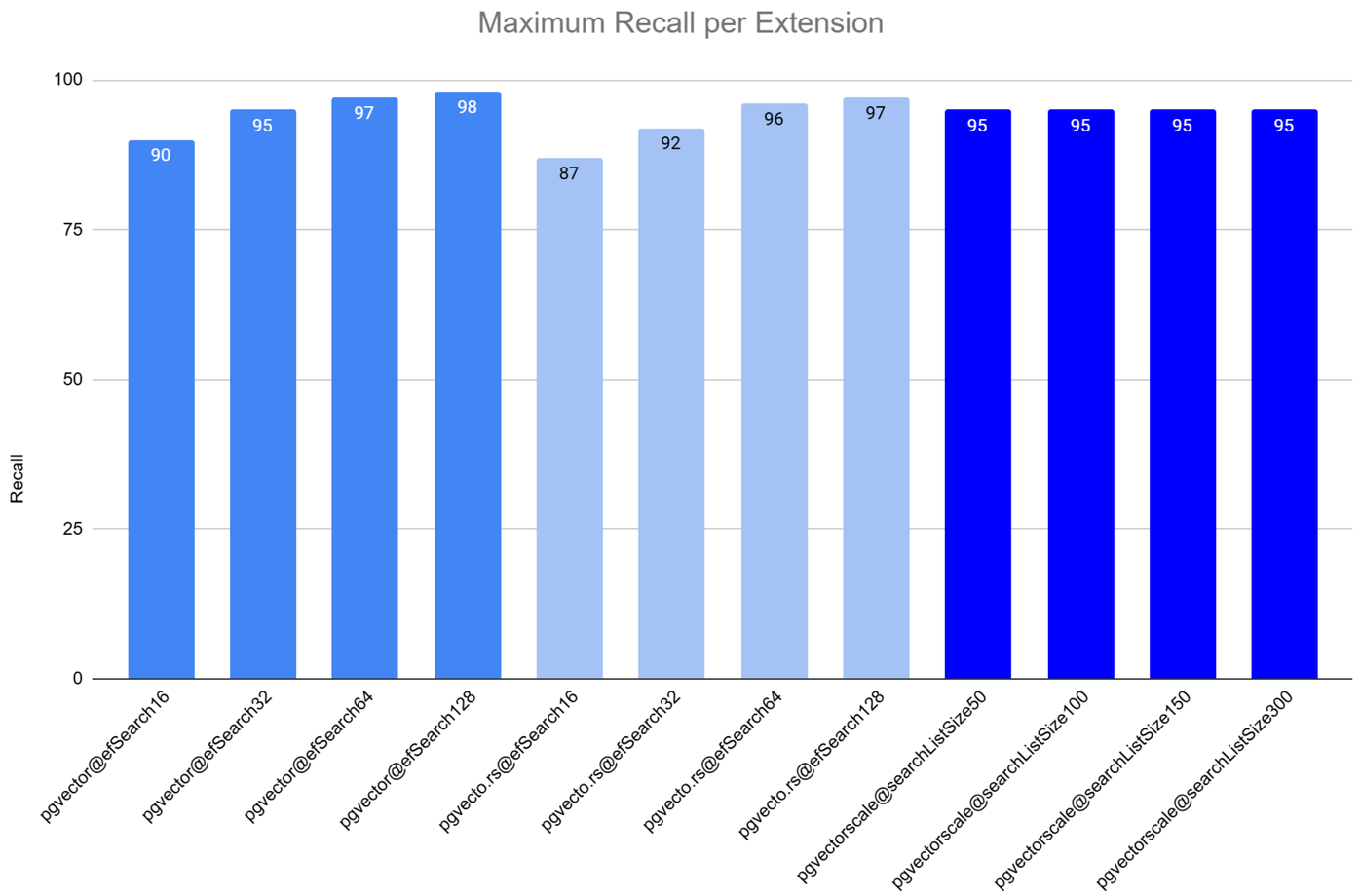
Goal: Illustrate performance differences between PostgreSQL vector extensions — not to judge or fully optimize each implementation.

		1M Cohre x 768				
PostgreSQL extension	**Index Type**	**Quantization**	**efConstruction**	**M**	**Index Size [GB]**	**Index Build Time [m]**
pgvector	HNSW	None	128	32	3.8	6
pgvecto.rs	HNSW	None	128	32	3.3	22
pgvectorscale	DiskANN	SBQ	128 (searchListSize)	64 (numNeighbors)	0.7	71

		10M Cohre x 768				
PostgreSQL extension	**Index Type**	**Quantization**	**efConstruction**	**M**	**Index Size [GB]**	**Index Build Time [h]**
pgvector	HNSW	**halfvec**	128	32	**20**	**2.3**
pgvecto.rs	HNSW	**scalar**	128	32	**41**	**11.3**
pgvectorscale	DiskANN	SBQ	128 (searchListSize)	64 (numNeighbors)	**7.4**	**26.2**

benchANT

Cohere 10Mx768 — Maximum Recall

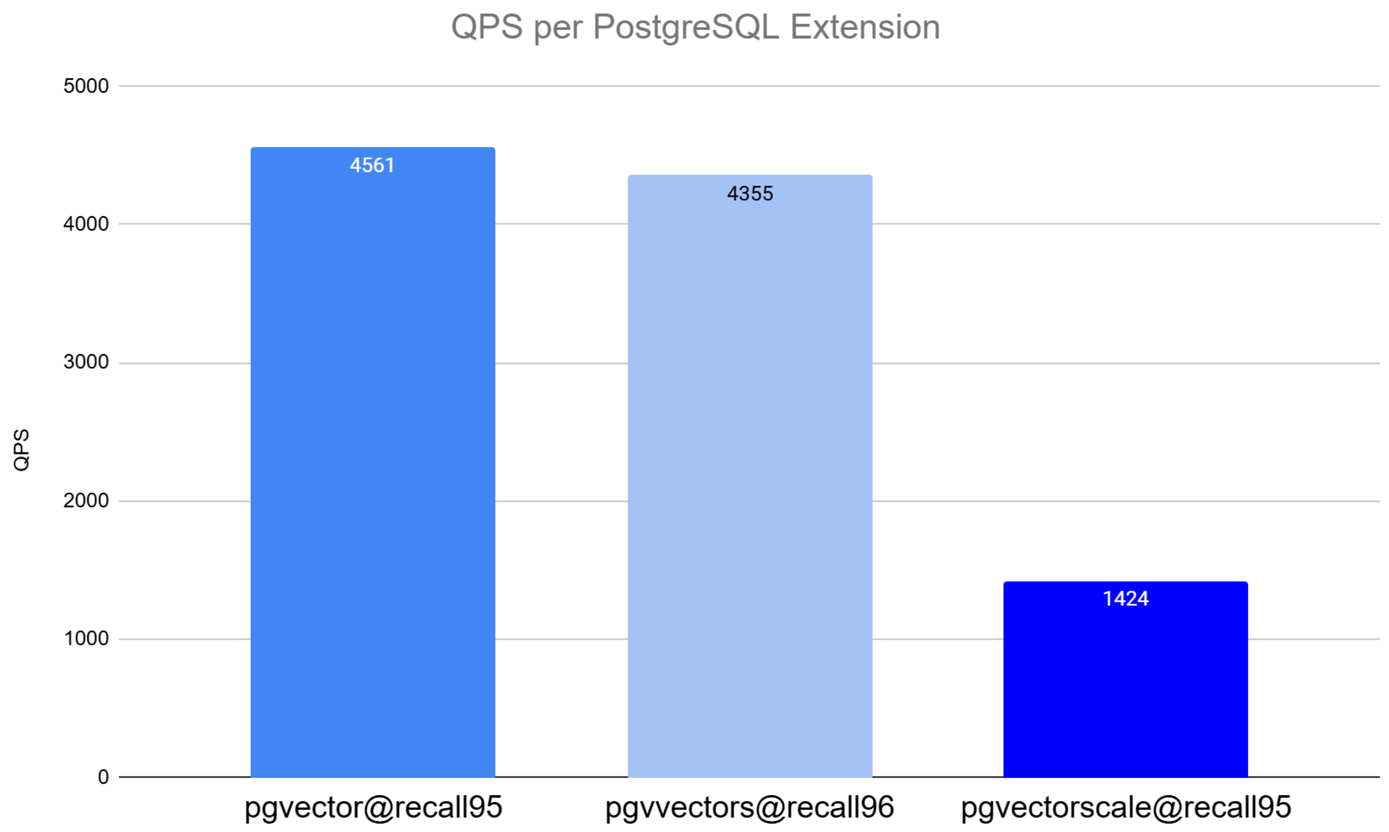


benchmark configuration

- PostgreSQL with `pgvector`, `pgvecto.rs`, `pgvectorscale`

benchANT

Cohere 10Mx768 — Maximum QPS



benchmark configuration

- PostgreSQL with `pgvector, pgvecto.rs, pgvectorscale`
- comparable recall levels
- threads = 32

benchANT

Where We Went Deep

- Vector search performance metrics
- QPS vs. recall trade-off
- Index build times
- Index memory requirements
- PostgreSQL vector search extensions and their general performance

benchANT

What We Skipped (for Now)

- Fine-tuning PostgreSQL for vector search use cases (e.g. with auto-tuning of DBTune or Releem)
- Fine-tuning vector search extensions
- HNSW index tuning
- Comparing different index types against each other, HNSW vs. IVF
- Filtering use cases
- Streaming use cases

benchANT

PostgreSQL Vector Search vs. Others

Goal: Illustrate performance differences between PostgreSQL as vector search database and other NoSQL and specialized Vector Databases without a comprehensive study nor fully optimize each implementation

- MongoDB Atlas with (vector) search nodes
- Zilliz Cloud (the DBaaS offer of the open source vector database Milvus)
- The following results are an Apple-to-Oranges comparison, so take the results with a grain of salt

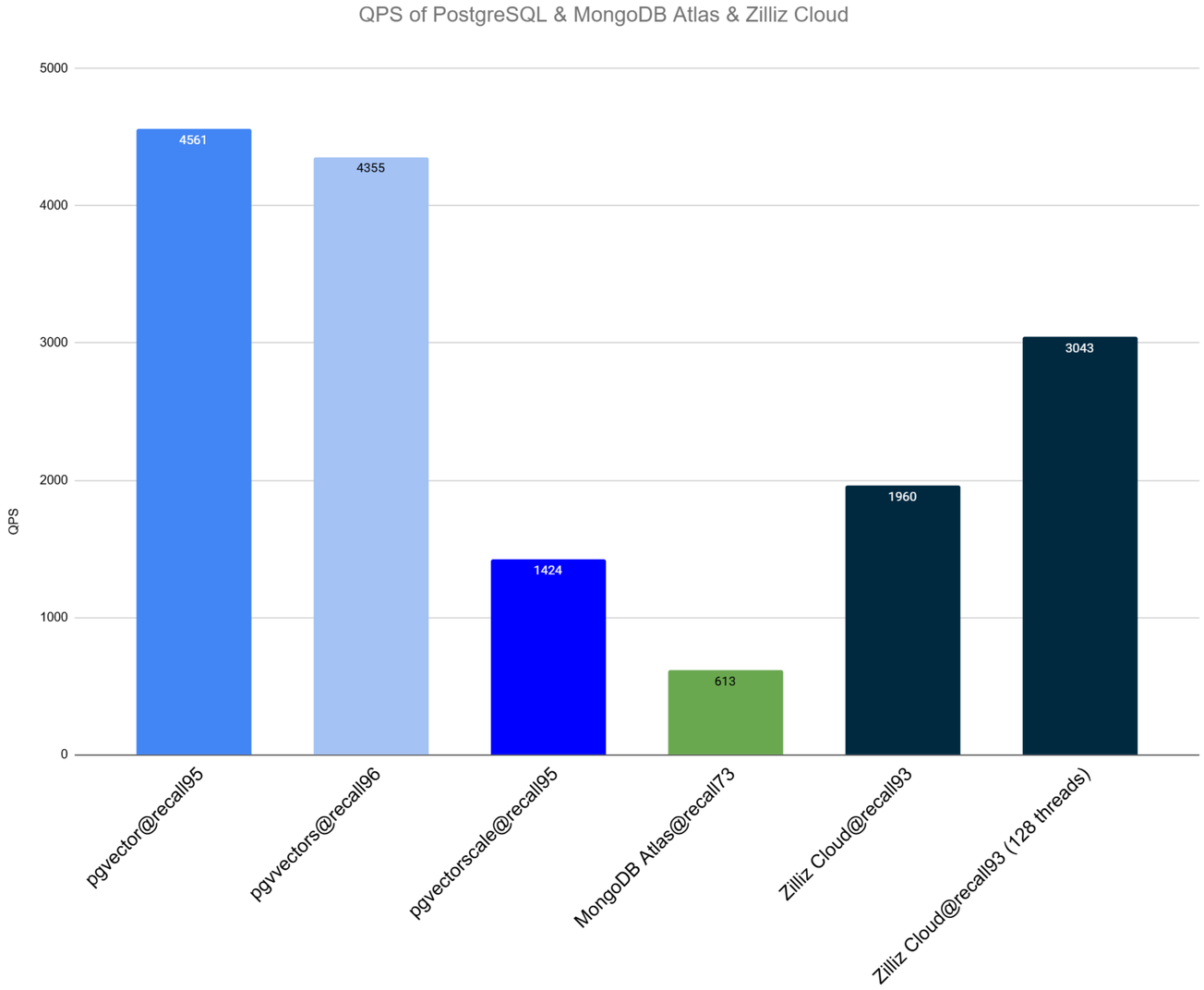
benchANT

PostgreSQL Vector Search vs. MongoDB Atlas vs. Zilliz Cloud

Database	Operation Type	Cloud	Cluster Size	Instance Size	Index Type
PostgreSQL with pgvector/pgvecto.rs /pgvectorscale	Self-managed@IaaS (non-production setup)	AWS	1 node	M7i.4xlarge (16 vCores/64 GB RAM / GP3)	HNSW with quantization / DiksANN with quantization
MongoDB Atlas	DBaaS	AWS	Replica set with 2 search nodes	M60 with default storage and S50 high-cpu search nodes (16 vcores / 32 GB RAM)	HNSW with quantization
Zilliz Cloud	DBaaS	AWS	8 CUs (based on Zilliz Cloud capacity recommender)	n/a	HNSW with quantization

benchANT

Cohere 10Mx768 — Maximum QPS — PostgreSQL vs. MongoDB vs. Zilliz



benchmark configuration

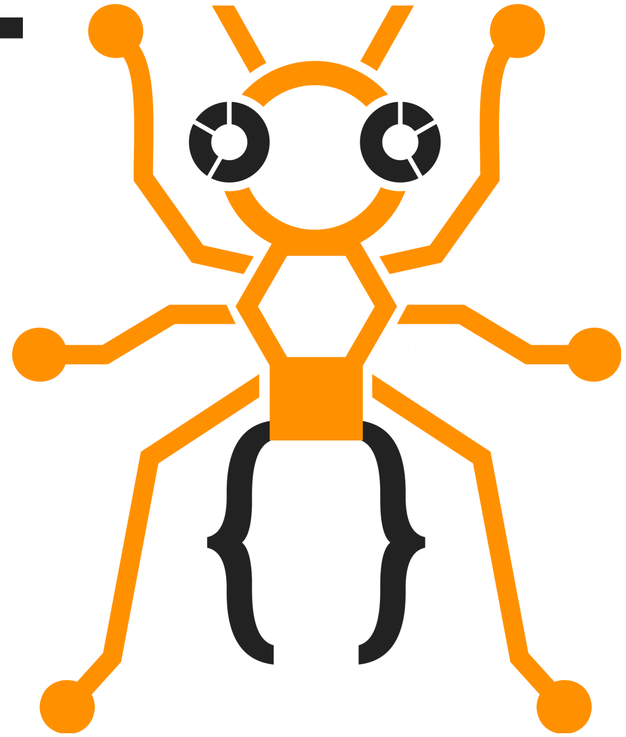
- PostgreSQL with `pgvector, pgvecto.rs, pgvectorscale`
- MongoDB with HNSW index
- Zilliz Cloud with HNSW_SQ index

benchANT

Conclusion

- PostgreSQL provides a rich ecosystem of vector search extension
- Vector search extensions differ in features and performance capabilities
- Vector search benchmarking requires the consideration of multiple dimensions (data set size, index type, concurrency, …)
- PostgreSQL with vector search extensions is able to compete with NoSQL and dedicated vector databases (at least in the presented data set sizes)
- Running a vector search use case with PostgreSQL in production besides the “standard” workloads requires careful planning

benchANT



Thank you!



Dr. Daniel Seybold
benchANT | daniel.seybold@benchant.com
DataScaleFail Newsletter | https://benchant.org/